

Building an End-to-End DevOps Project on AWS using Terraform, Kubernetes, Jenkins (CI/CD), GitOps, ArgoCD with Full Prometheus Monitoring & Grafana Visualization

AWS

Terraform

Jenkins

Docker

ArgoCD

Kubernetes / EKS

Prometheus

Grafana

GitOps

In this project, I build a full end-to-end DevOps project on AWS with GitOps workflow. The entire infrastructure was provisioned using Terraform, with state and lock management in AWS S3. The application code is managed on GitHub, and a webhook triggers Jenkins to clone the repo, build a Docker image, push it to Amazon ECR, and update a GitOps-managed repo. ArgoCD watches this repo and automatically deploys to Amazon EKS, using Kubernetes Deployments for app pods and StatefulSets for database pods, backed by Amazon EFS for persistent storage. External access is routed via an Ingress Controller using AWS ALB, secured by AWS Certificate Manager (ACM) and Route 53 for DNS. The entire stack is monitored by Prometheus and visualized through Grafana, with RBAC controlling access and alerts sent via email for any failures in Jenkins pipelines or unhealthy services.

The tech stack includes Terraform, GitHub, Jenkins, Docker, ArgoCD, Helm, Kubernetes, AWS EKS, ECR, EFS, ALB, ACM, Route 53, Prometheus, Grafana, ConfigMap, Secrets, RBAC, and more — delivering a robust, automated, scalable, and secure DevOps pipeline.

Project Overview

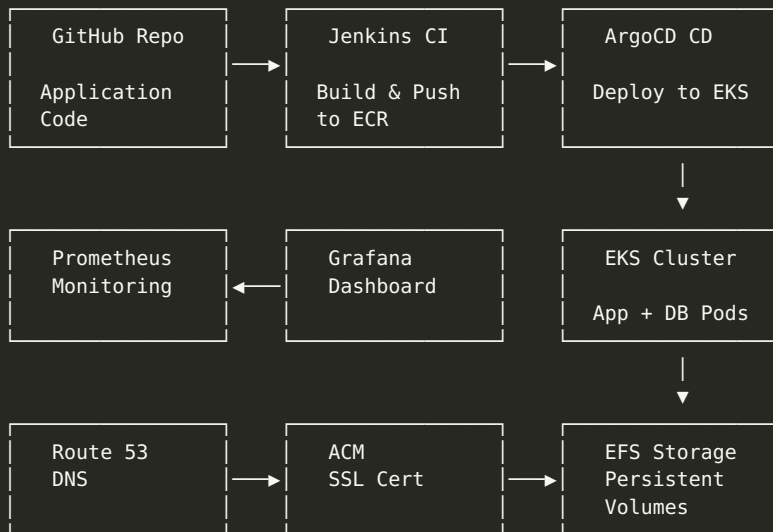
This project implements a complete GitOps workflow on AWS with automated infrastructure provisioning, continuous integration/deployment, and comprehensive monitoring. The application is a Python Django web application with MySQL database, deployed on Amazon EKS with persistent storage using EFS.

Key Features:

- **Infrastructure as Code:** Complete AWS infrastructure managed with Terraform
- **CI/CD Pipeline:** Jenkins automated build and deployment pipeline
- **GitOps Workflow:** ArgoCD for Kubernetes deployment automation
- **Container Orchestration:** Kubernetes with EKS for scalable deployments
- **Persistent Storage:** EFS-backed storage for database and application data
- **Load Balancing:** AWS ALB with SSL termination via ACM
- **Monitoring:** Prometheus metrics collection with Grafana visualization
- **Security:** RBAC, secrets management, and network security groups

Architecture

DIAGRAM



Tech Stack

Infrastructure & DevOps

- **Terraform** - Infrastructure as Code
- **AWS EKS** - Kubernetes cluster
- **AWS ECR** - Container registry
- **AWS EFS** - Persistent storage
- **AWS ALB** - Load balancer
- **AWS ACM** - SSL certificates
- **Route 53** - DNS management

CI/CD & GitOps

- **Jenkins** - CI/CD pipeline
- **ArgoCD** - GitOps deployment
- **Docker** - Containerization
- **GitHub** - Source code & GitOps repo

Monitoring & Observability

- **Prometheus** - Metrics collection
- **Grafana** - Visualization & dashboards
- **AlertManager** - Alert management

Application

- **Python Django** - Web application
- **MySQL** - Database
- **Kubernetes** - Container orchestration

Prerequisites

Before starting, ensure you have the following:

Required Software

- [Terraform](#) (v1.0+)
- [AWS CLI](#) (v2.0+)
- [kubectl](#) (v1.24+)
- [Docker](#) (v20.0+)
- [Git](#) (v2.30+)

AWS Account Requirements

- AWS account with appropriate permissions
- S3 bucket for Terraform backend (create manually first)
- Domain name for Route 53 (optional but recommended)

GitHub Requirements

- GitHub account
- Personal access token with repo permissions
- Two repositories:
 - Main application repository
 - GitOps repository for Kubernetes manifests

Quick Start

1. Clone the Repository

BASH

```
git clone https://github.com/yourusername/Building-an-End-to-End-DevOps-Project-on-AWS.git
cd Building-an-End-to-End-DevOps-Project-on-AWS
```

2. Configure AWS Credentials

BASH

```
aws configure
# Enter your AWS Access Key ID, Secret Access Key, and default region (ap-southeast-1)
```

3. Create S3 Backend Bucket

BASH

```
aws s3 mb s3://terraform-devops-backend-file --region ap-southeast-1
```

4. Deploy Infrastructure

BASH

```
cd Terraform
terraform init
terraform plan
terraform apply
```

5. Configure Jenkins & ArgoCD

Follow the detailed setup guide below for complete configuration.

Detailed Setup Guide

Phase 1: Infrastructure Setup

Step 1: Configure Terraform Backend

1. Update the S3 bucket name in `Terraform/provider.tf`:

TERRAFORM / HCL

```
terraform {
  backend "s3" {
    bucket = "your-terraform-backend-bucket"
    region = "ap-southeast-1"
    key = "terraform.tfstate"
    encrypt = true
    use_lockfile = true
  }
}
```

Step 2: Deploy AWS Infrastructure

BASH

```
cd Terraform
terraform init
terraform plan
terraform apply -auto-approve
```

This will create:

- VPC with public/private subnets
- EKS cluster
- ECR repository
- EFS file system
- EC2 instance for Jenkins
- Route 53 hosted zone
- ACM certificate

Step 3: Configure kubectl

BASH

```
aws eks update-kubeconfig --region ap-southeast-1 --name your-eks-cluster-name
kubectl get nodes
```

Phase 2: Jenkins Setup

Step 1: Connect to Jenkins Server

BASH

```
# Get the public IP of your Jenkins EC2 instance
aws ec2 describe-instances --filters "Name=tag:Name,Values=Jenkins-Server" --query
'Reservations[].Instances[].PublicIpAddress' --output text

# SSH to the server
ssh -i your-key.pem ubuntu@<jenkins-public-ip>
```

Step 2: Install Jenkins

Follow the installation guide in [Install and Configuration/Jenkins, Docker and AWS CLI installation.txt](#)

Step 3: Configure Jenkins Credentials

1. Access Jenkins UI (<http://jenkins-public-ip:8080>)
2. Add GitHub token as credential:
 - Go to Manage Jenkins > Credentials > System > Global credentials
 - Add new credential:
 - Kind: Secret text
 - ID: GITHUB_TOKEN
 - Secret: Your GitHub personal access token

Step 4: Create Jenkins Pipeline

1. Create new Pipeline job
2. Configure webhook trigger from GitHub
3. Use the Jenkinsfile from [Jenkins_cicd/Jenkinsfile](#)

Phase 3: ArgoCD Setup

Step 1: Install ArgoCD

BASH

```
kubectl create namespace argocd
kubectl apply -n argocd -f https://raw.githubusercontent.com/argoproj/argo-cd/stable/manifests/install.yaml
```

Step 2: Access ArgoCD UI

BASH

```
kubectl port-forward svc/argocd-server -n argocd 8080:443
# Access https://localhost:8080
# Default username: admin
# Get password: kubectl -n argocd get secret argocd-initial-admin-secret -o jsonpath="{.data.password}" |
base64 -d
```

Step 3: Create ArgoCD Application

1. Create application pointing to your GitOps repository
2. Set path to [Kubernetes with ArgoCD/](#)
3. Enable auto-sync

Phase 4: Application Deployment

Step 1: Update Configuration

1. Update image references in Kubernetes manifests
2. Configure database credentials in secrets
3. Update domain name in ingress configuration

Step 2: Deploy Application

BASH

```
kubectl apply -f "Kubernetes with ArgoCD/"
```

Phase 5: Monitoring Setup

Step 1: Install Prometheus

BASH

```
kubectl create namespace monitoring  
kubectl apply -f "Install and Configuration/prometheus yml file.txt"
```

Step 2: Install Grafana

BASH

```
kubectl apply -f "Install and Configuration/Grafana installation.txt"
```

Step 3: Configure Dashboards

1. Access Grafana (port-forward or ingress)
2. Import Prometheus as data source
3. Import monitoring dashboards

Monitoring & Alerts

Prometheus Configuration

- Metrics collection from Kubernetes pods
- Custom metrics for application health
- Alert rules for critical failures

Grafana Dashboards

- Kubernetes cluster overview
- Application performance metrics
- Database monitoring
- Jenkins pipeline status

Alert Configuration

- Email alerts for pipeline failures
- Service health monitoring
- Resource utilization alerts

Configuration Files

Key Configuration Files:

- `Terraform/` - Infrastructure as Code
- `Jenkins_cicd/Jenkinsfile` - CI/CD pipeline
- `Kubernetes with ArgoCD/` - Kubernetes manifests
- `Install and Configuration/` - Setup scripts and configs

Environment Variables:

BASH

```
AWS_REGION=ap-southeast-1
AWS_ACCOUNT_ID=your-account-id
GITHUB_TOKEN=your-github-token
```

Troubleshooting

Common Issues:

1. Terraform Backend Issues

BASH

```
# If S3 backend doesn't exist
aws s3 mb s3://terraform-devops-backend-file --region ap-southeast-1
```

2. EKS Connection Issues

BASH

```
# Update kubeconfig
aws eks update-kubeconfig --region ap-southeast-1 --name your-cluster-name
```

3. Jenkins Pipeline Failures

- Check GitHub token permissions
- Verify ECR repository access
- Ensure Docker is running on Jenkins server

4. ArgoCD Sync Issues

- Verify GitOps repository access
- Check Kubernetes manifest syntax
- Review ArgoCD application logs

5. Application Deployment Issues

BASH

```
# Check pod status
kubectl get pods -n default

# Check pod logs
kubectl logs <pod-name>

# Check events
kubectl get events --sort-by='.lastTimestamp'
```

Useful Commands:

BASH

```
# Check EKS cluster status
aws eks describe-cluster --name your-cluster-name --region ap-southeast-1

# List ECR repositories
aws ecr describe-repositories --region ap-southeast-1

# Check EFS mount targets
aws efs describe-mount-targets --file-system-id your-efs-id

# Monitor Jenkins logs
sudo journalctl -u jenkins -f

# Check ArgoCD application status
kubectrl get applications -n argocd
```

Contributing

1. Fork the repository
2. Create a feature branch (`git checkout -b feature/amazing-feature`)
3. Commit your changes (`git commit -m 'Add amazing feature'`)
4. Push to the branch (`git push origin feature/amazing-feature`)
5. Open a Pull Request

License

This project is licensed under the MIT License - see the [LICENSE](#) file for details.

Support

For support and questions:

- Create an issue in the GitHub repository
- Check the troubleshooting section above
- Review the configuration files for reference

Note: This project is for educational and demonstration purposes. Please review and modify configurations according to your specific requirements and security policies before using in production environments.